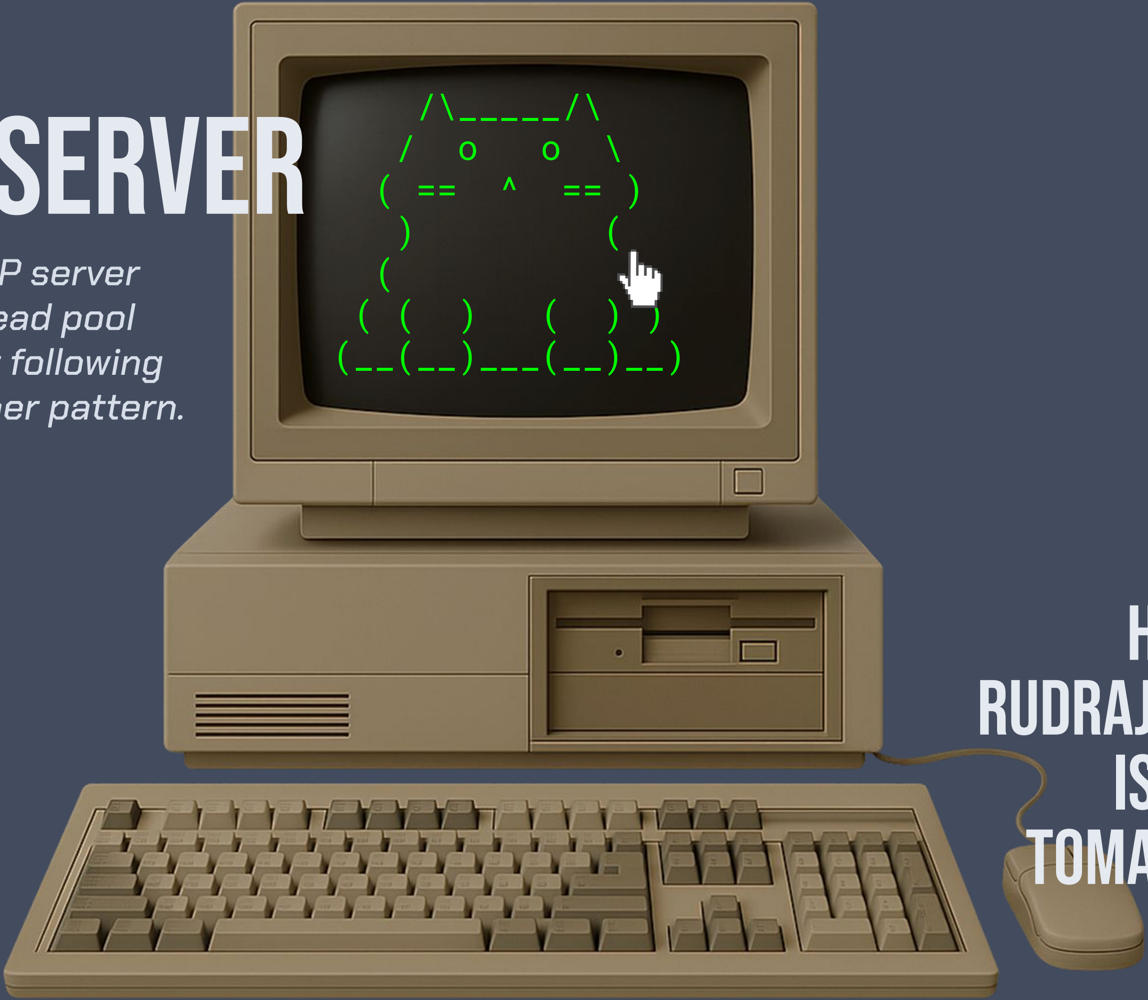
TEAM 0101

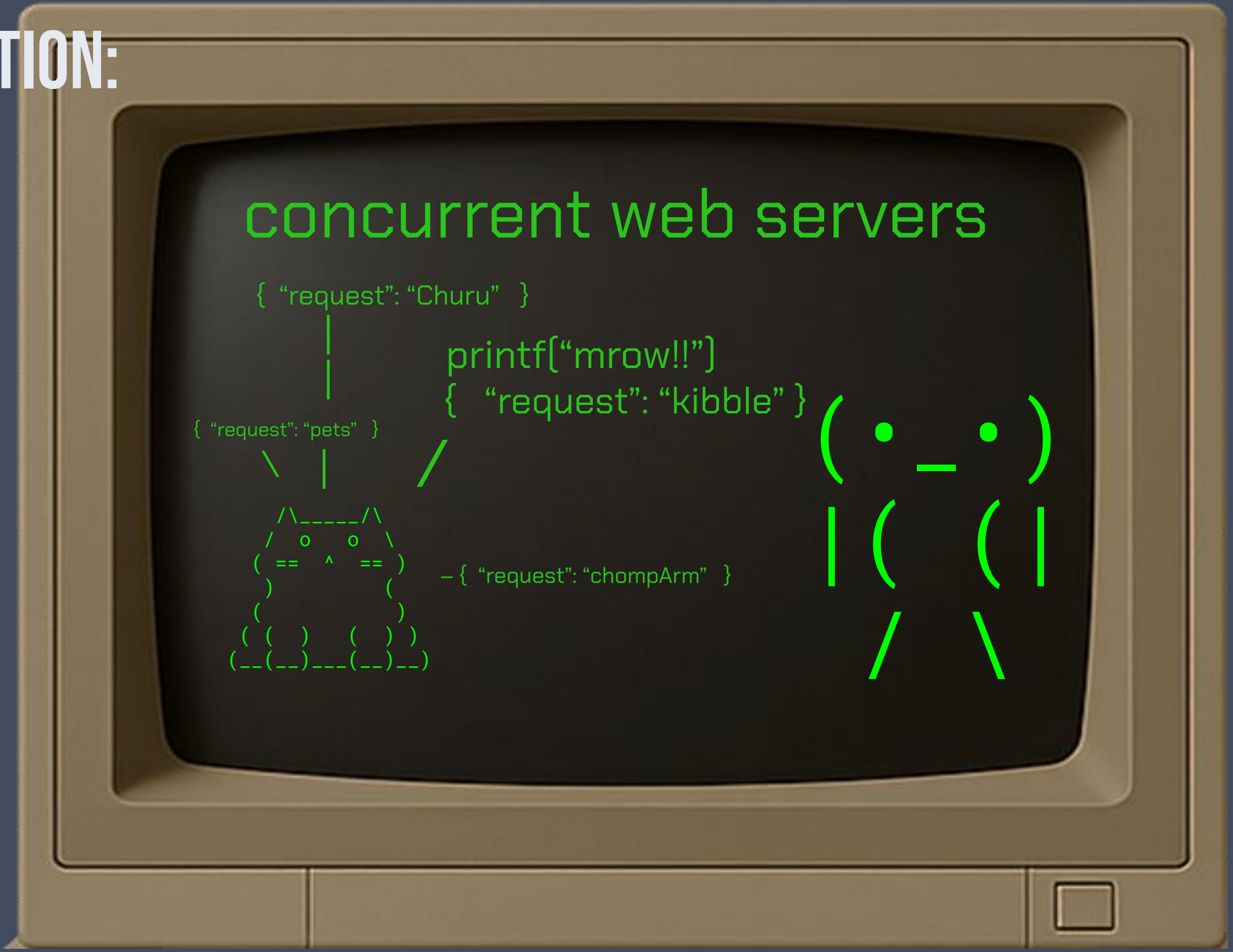
G.5 WEB SERVER

A multi-threaded HTTP server using a fixed-size thread pool and a bounded buffer following the producer-consumer pattern.

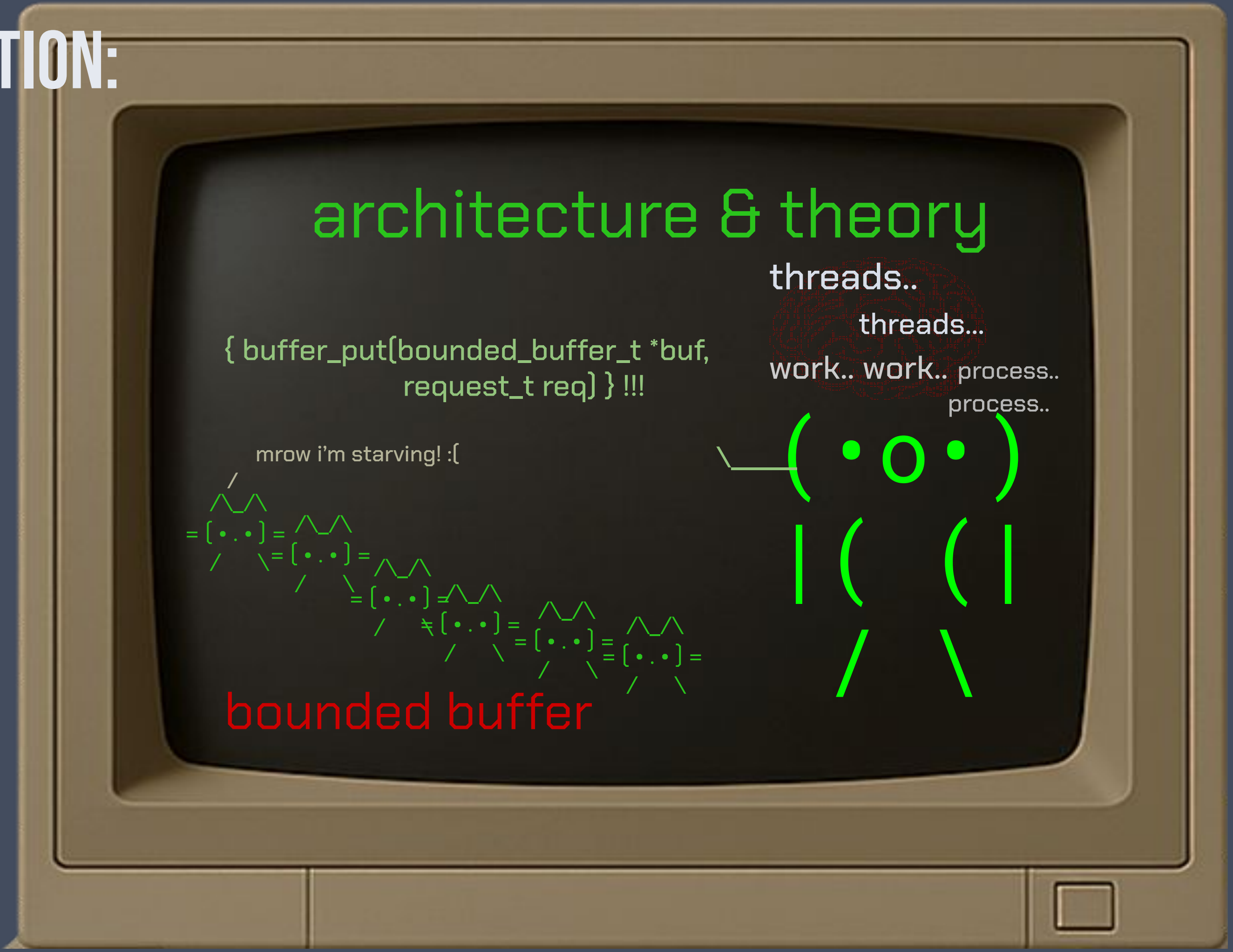


HELENA ZHUO
RUDRAJIT BANERJEE
ISHIT ARHATIA
TOMAS COLORADO

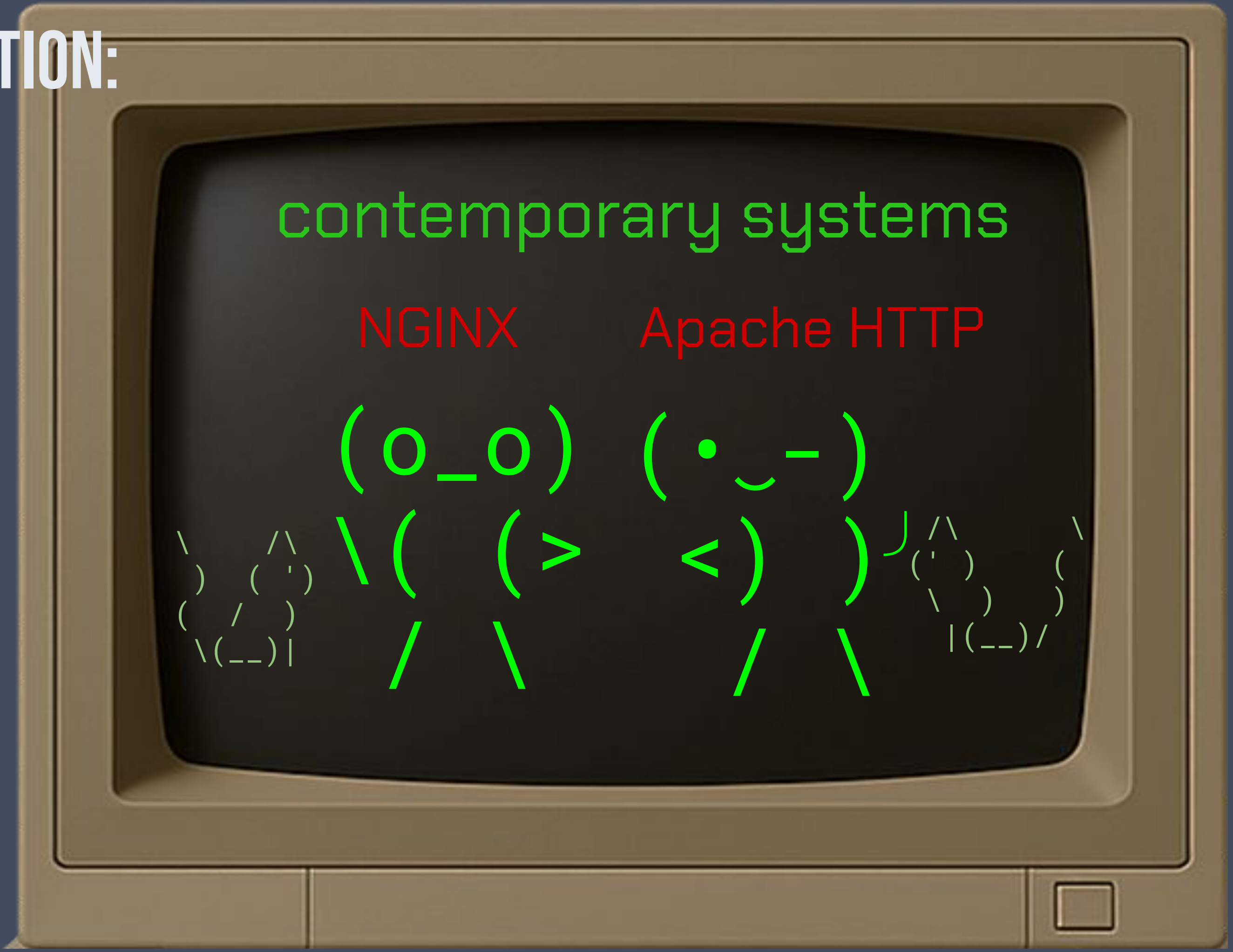
INTRODUCTION:



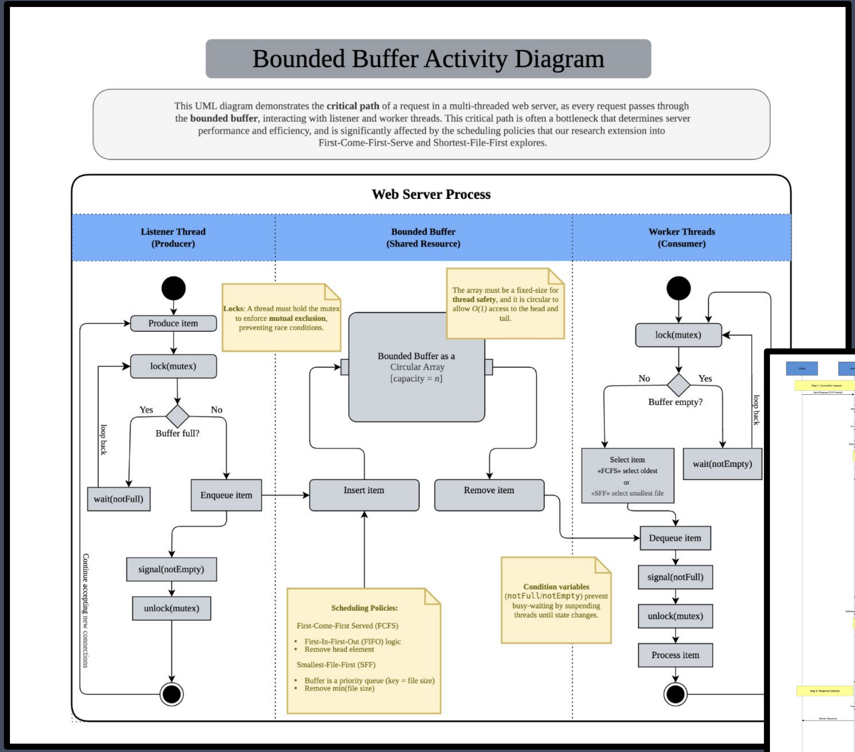
INTRODUCTION:

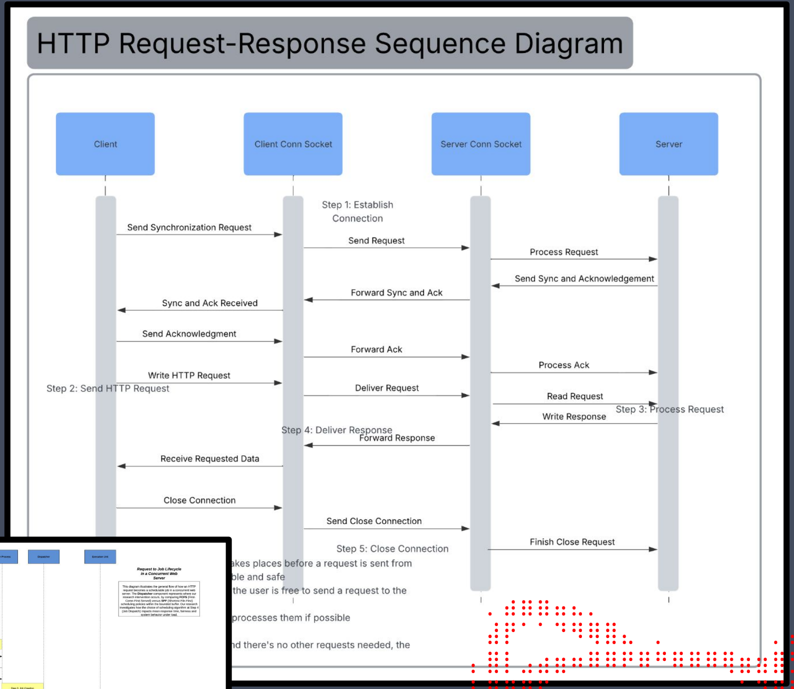


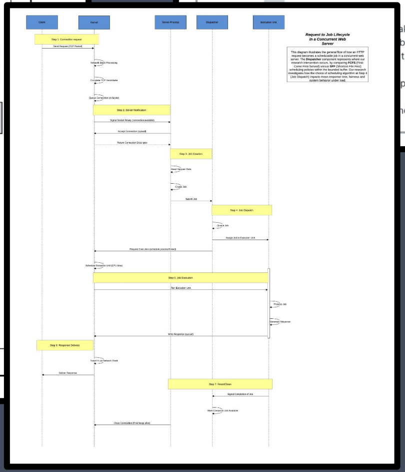
INTRODUCTION:



IMPLEMENTATION DESIGN: UML DIAGRAMS







IMPLEMENTATION DESIGN:
UML DIAGRAMS

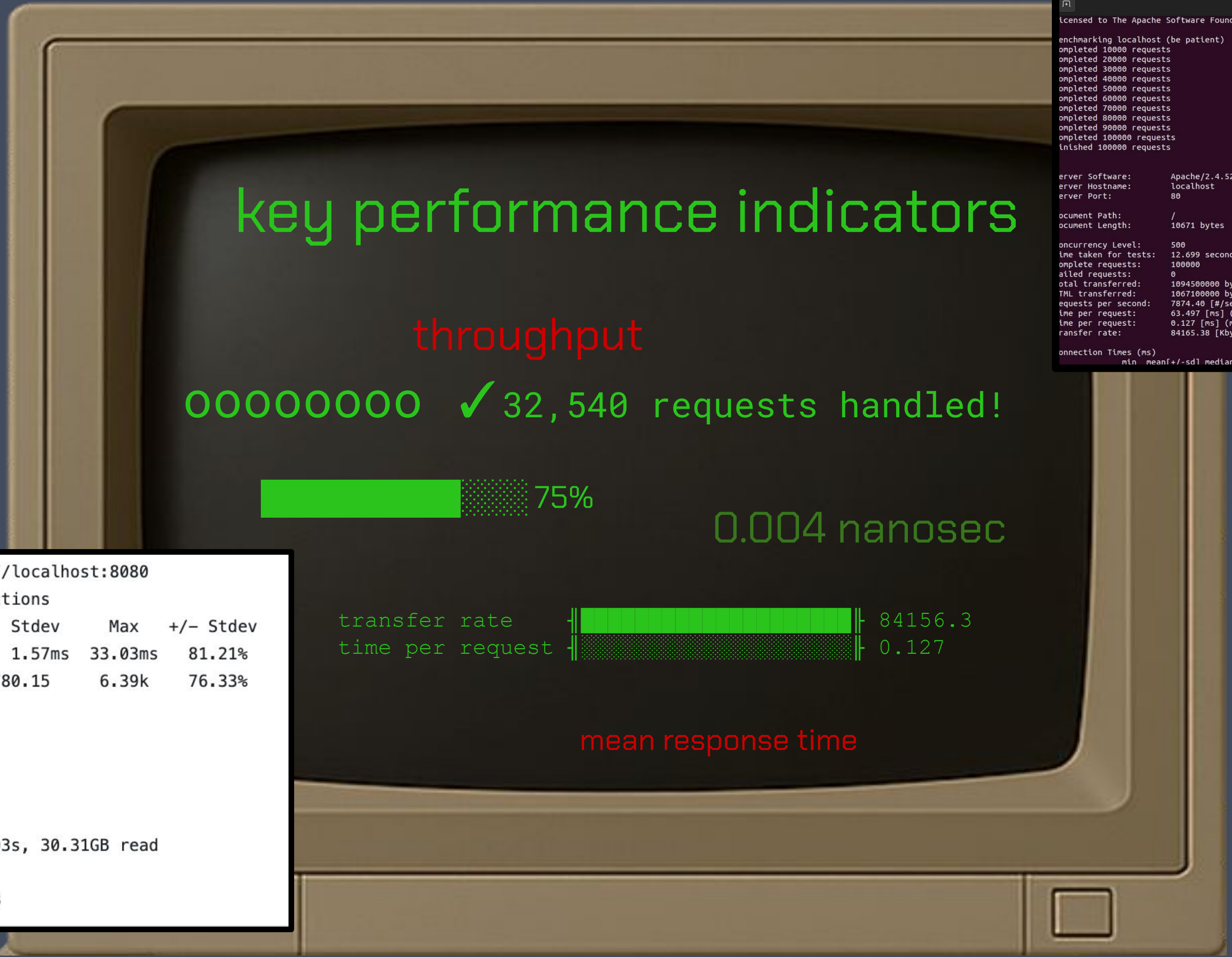
experiencing.. heavy loads..

```
{
    var concurrency_level = 100;
    const hunger = 100;
    const request = “kibble kibble kibble”
    var PATH = $ROOT/myBelly
}
```

PROBLEM:


workloads of varying sizes
uniform small
uniform large
heavy-tailed

PROBLEM:




```
rudra@CS5600VM: ~/Desktop
icensed to The Apache Software Foundation, http://www.apache.org/

enchmarking localhost (be patient)
ompleted 10000 requests
ompleted 20000 requests
ompleted 30000 requests
ompleted 40000 requests
ompleted 50000 requests
ompleted 60000 requests
ompleted 70000 requests
ompleted 80000 requests
ompleted 90000 requests
ompleted 100000 requests
inished 100000 requests


erver Software:        Apache/2.4.52
erver Hostname:        localhost
erver Port:            80

ocument Path:          /
ocument Length:        10671 bytes

oncurrency Level:      500
ime taken for tests:   12.699 seconds
omplete requests:      100000
ailed requests:        0
otal transferred:      1094500000 bytes
TML transferred:       1067100000 bytes
equests per second:    7874.40 [#/sec] (mean)
ime per request:       63.497 [ms] (mean)
ime per request:       0.127 [ms] (mean, across all concurrent requests)
ransfer rate:          84165.38 [Kbytes/sec] received

onnection Times (ms)
              min  mean[+/-sd] median   max
```

```
Running 30s test @ http://localhost:8080
  2 threads and 25 connections
  Thread Stats   Avg      Stdev     Max   +/- Stdev
    Latency     2.33ms    1.57ms  33.03ms   81.21%
    Req/Sec     5.00k   780.15     6.39k    76.33%
  Latency Distribution
     50%    2.06ms
     75%    2.91ms
     90%    3.94ms
     99%    8.08ms
  298698 requests in 30.03s, 30.31GB read
Requests/sec:   9945.82
Transfer/sec:      1.01GB
```

SOLUTION:
SIZE-BASED SCHEDULING

```
1     /\_/\
   =( •.• )=  a small mrow 4kb
     /   \

      /\_/\
2  =( •.• )=  a big mrow 1MB!!  —- get in line, felines.
     /   \

3     /\_/\
   =( •.• )=  a small mrow 4kb
     /   \

4     /\_/\
   =( •.• )=  a medium mrow.. between 4kb and 1MB
     /   \
```

FCFS
First-Come-First-Served

VS

```
                              /\_/\
smallest mrows first! —-   =( •.• )=  a small mrow 4kb   1
                        \     /   \
                         \
                              /\_/\
                           =( •.• )=  a small mrow 4kb   2
                              /   \

                              /\_/\
                           =( •.• )=  a small mrow 4kb   3
                              /   \

                            /\_/\
                         =( •.• )=  a big mrow 1MB!!     4
                            /   \
```

SFF
Shortest-File-First

CONCURRENT WEB SERVER:

implementation - core

```
src/
├── main.c                  # Entry point
├── bounded_buffer.c  # mutex, condvars,
│                            FCFS/SFF dequeue
├── thread_pool.c       # init, worker creation
└── http.c                  # parsing, status
                              codes, file send
```

Team 0101's Concurrent Web Server

By Helena Zhuo, Rudrajit Banerjee, Ishit Arhatia, Tomas Colorado

A multi-threaded HTTP server using a fixed-size thread pool and a bounded buffer, following the producer-consumer pattern.

See our Github for more information: README.md Research Extension Glossary.md Benchmarking Rig Bibliography

Workload Simulator

Simulate a Workload

implementation - core (continued)

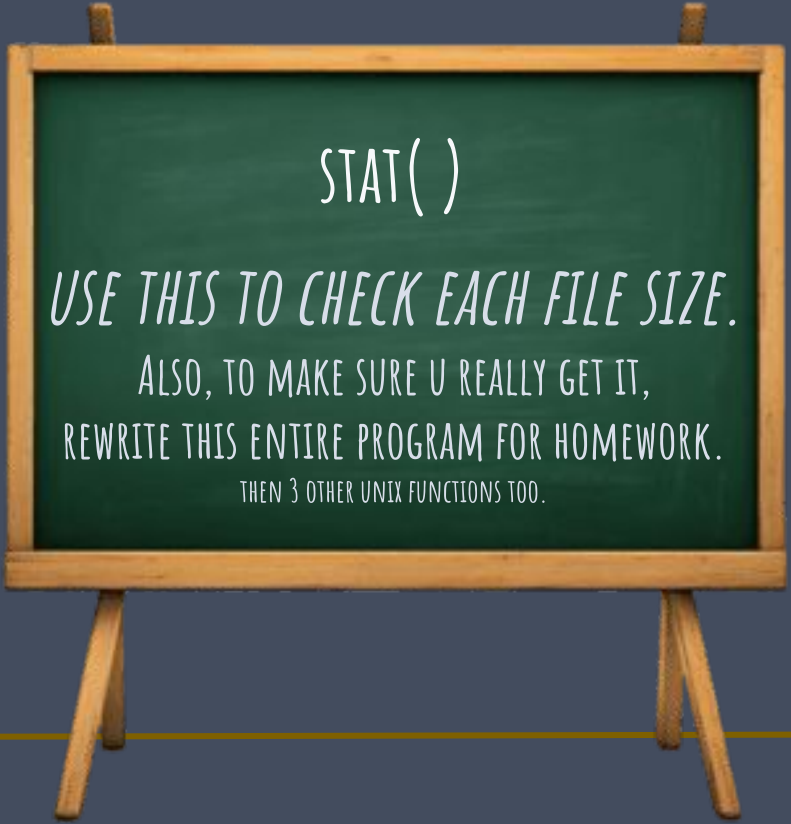
```
./server

======= CONFIGURATION =======
Port: 8080
Threads: 4
Buffer Capacity: 16
Scheduling Policy: FCFS
Thread Pool Initialization: 4 workers spawned

======= SERVER RUNNING =======
Ctrl+click (cmd+click on Mac) to open in your browser: http://localhost:8080
```

CONCURRENT WEB SERVER: IMPLEMENTATION - RESEARCH EXTENSION

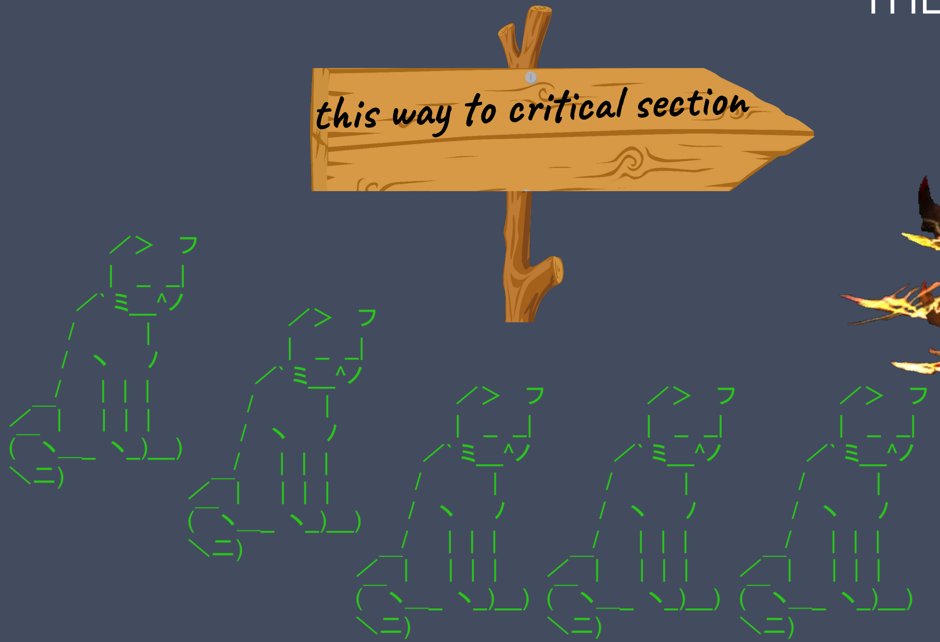
welcome to class.

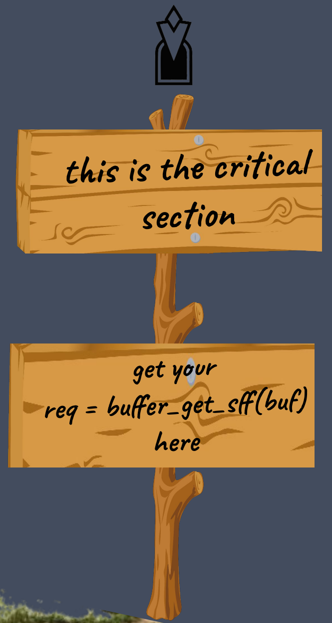
this is very important.

```
  /\_/\
 ( o.o )
  > ^ <
 /  ~  \
(_/---\_)
```



CONCURRENT WEB SERVER:
IMPLEMENTATION - RESEARCH EXTENSION





IMPLEMENTATION: DESIGN CHOICES

CONFIGURABILITY

RUNTIME FLAGS

port

default: 8080

thread count

default: 4

buffer size

default: 16

scheduling policy

default: FCFS

MEASUREMENTS: METHODOLOGY & SOURCES

POLLING

Welcome to the official GNU Netcat project homepage

The GNU Netcat project

LOAD GENERATOR

NETCAT, WRK & LUA

Will Glozer
wg

wrk Public

Modern HTTP benchmarking tool

C ☆ 40.2k 3k

MEASUREMENTS:
ENVIRONMENT & DESIGN

rigor & reproducibility

VM SETUP

- ORACLE VIRTUALBOX
- UBUNTU 22.04 LTS
- 4 VCPUS, 8 GB RAM
- GCC 11.4
- WRK 4.1.03

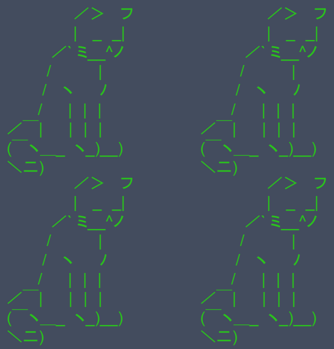
ANALYSIS: PERFORMANCE

Workload	Concurrency	FCFS	SFF	Gain
Small (4 KB)	10	0.56	0.56	~0%
	100	3.55	4.10	-15.5%
Large (1 MB)	10	6.64	5.84	+12.0%
	100	56.34	60.66	-7.7%
Heavy-tailed	10	1.21	1.13	+6.6%
	100	8.79	10.00	-13.8%

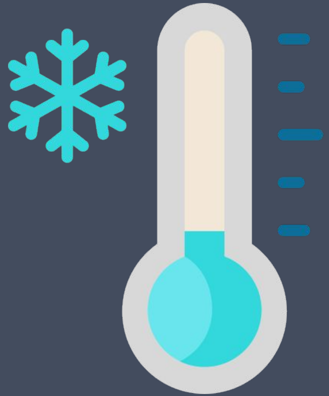
ANALYSIS:
PERFORMANCE - WHY?

3 MAIN FACTORS!

THE SFF SCAN
(OVERHEAD)



ALWAYS BUSY WORKER
THREADS...
(THREAD SATURATION)



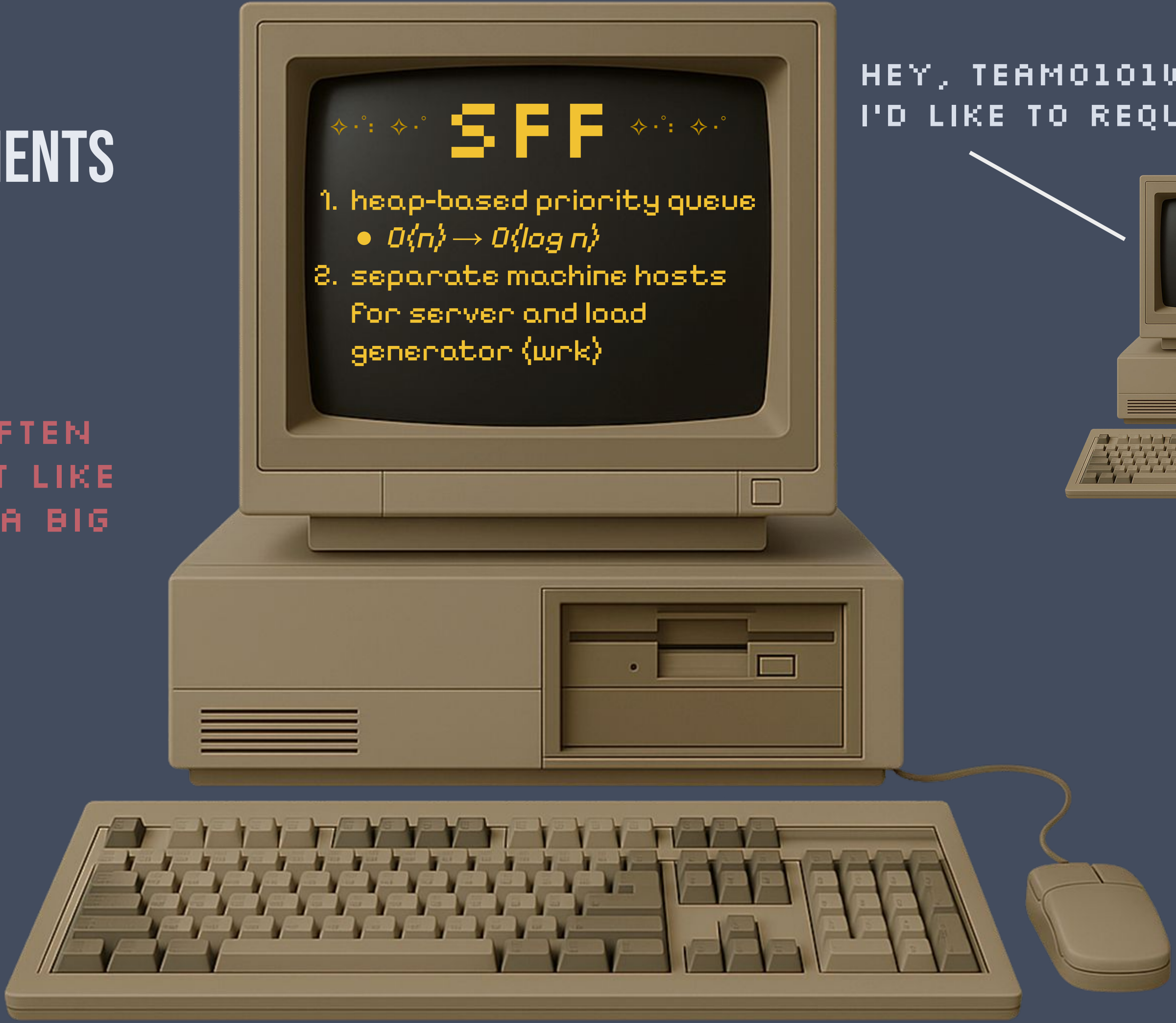
SERVER RESTART
VARIANCE
(COLD-START CACHE STATE)

ANALYSIS:
IMPACT OF SCHEDULING POLICIES

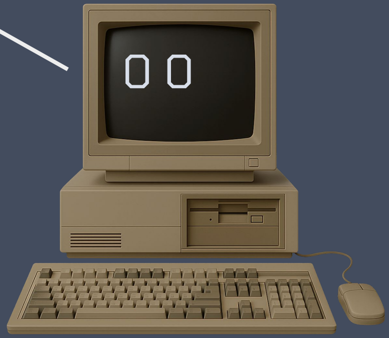
Yeah, and only under low currency.. we don't even want to talk about under high contention...

The improvements are...
MARGARINE-nally beneficial...

CONCLUSION & FUTURE IMPROVEMENTS



HEY, TEAM0101WEBSERVER. I'D LIKE TO REQUEST(FILE).



IT'S NOT OFTEN THAT A CAT LIKE ME MAKES A BIG REQUEST..

```
  /\_____/\
 /  o   o  \
( ==  ^  == )
 )         (
(           )
( (  )   (  ) )
(__(__)___(__)__)
```